10.1.2 BDD builds on established TDD practices

The core practices we're looking at in this chapter are essentially those of TDD, or are deeply rooted in TDD. Indeed, experienced TDD practitioners have been using these techniques for a long time. In many respects, BDD has formalized the way many successful TDD practitioners have been doing things, emphasizing a few key aspects of advanced TDD practice that make the technique more effective. These practices include

- Using outside-in development to ensure that the code you write is delivering real business value
- Using a shared (or "ubiquitous") domain language to encourage closer collaboration and understanding within the team
- Using examples to describe this behavior more clearly
- Describing and specifying the behavior of the system at both a high and a more detailed level

You'll see more of these concepts in the rest of the chapter.

10.1.3 BDD unit-testing tools are there to help

The result of this formalization is that a number of more BDD-flavored unit-testing tools have emerged over recent years that make these techniques easier and more intuitive to practice. Tools like RSpec, NSpec, Spock, and Jasmine have flourished over recent years. While there's no obligation to use these tools (many of our examples will be written simply using JUnit), they can make it easier to write more concise, more expressive, low-level executable specifications.

In the remainder of this chapter, you'll learn how to apply the BDD principles you've seen so far to design and deliver working code. Or, if you prefer, you'll learn to practice TDD with a BDD flavor. If you don't have any experience with TDD, don't worry. I'll introduce all the key concepts as we go.

10.2 Going from acceptance criteria to implemented features

In this section we'll walk through a typical BDD workflow, going from high-level automated acceptance criteria to application code. We'll keep the requirements simple, so that we can focus on the process. But before we start, let's take a look at what we mean by "outside-in" development.

10.2.1 BDD favors an outside-in development approach

One of the core practices used in BDD is *outside-in development*. This involves using acceptance criteria to drive the implementation details you need to build business features. You start with the outcomes you expect and use these outcomes to determine what code you need to write.

The process typically iterates over the following steps:

1. Start with a high-level acceptance criterion that you want to implement.
2. Automate the acceptance criterion as pending scenarios, breaking the acceptance criterion into smaller steps.